

Language Model Post-Training

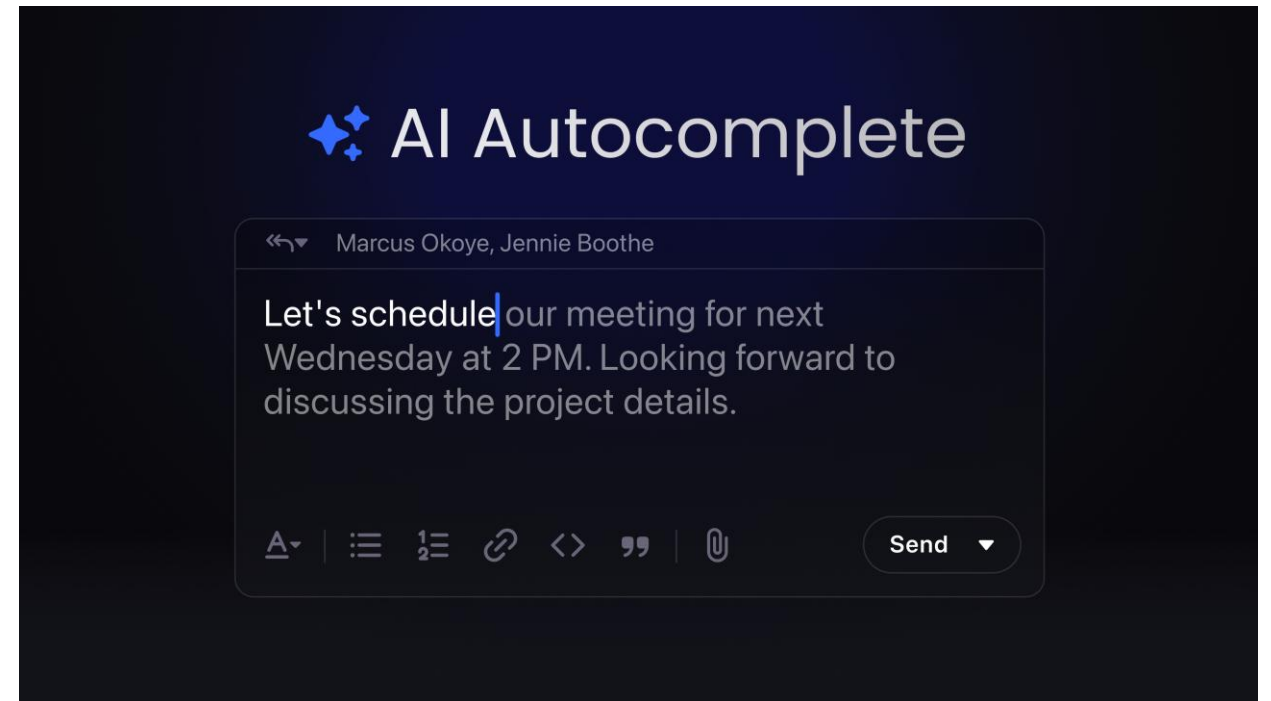
Robin Jia & Xuezhe Ma
USC CSCI 544, Spring 2026
February 26, 2026

Previously: Autoregressive Pre-Training

- Training example: “Neural networks are powerful learners”
- Want to maximize $P(\text{“Neural networks are powerful learners”})$
- MLE: Take log and decompose by chain rule:
$$\begin{aligned} &\log P(\text{“Neural”}) \\ &+ \log P(\text{“networks”} \mid \text{“Neural”}) \\ &+ \log P(\text{“are”} \mid \text{“Neural networks”}) \\ &+ \log P(\text{“powerful”} \mid \text{“Neural networks are”}) + \dots \end{aligned}$$
- Decomposes into a bunch of **next-word-classification** problems
- Backpropagation + gradient descent to minimize loss

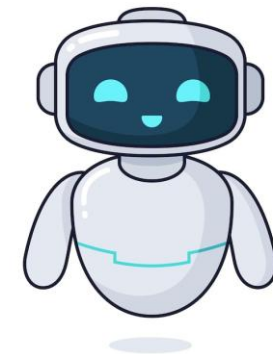
Pre-Training is Only the First Step

- Pre-trained LM is a very good “autocomplete” system
- Model has learned a lot of capabilities & knowledge that were required to do autocomplete well
 - Enabled by large pre-training dataset
- Downside: You’ve trained the model to imitate the average internet post
 - High quantity of data, low quality!



Previously: Fine-Tuning

- How do we get the model to exhibit a specific behavior?
 - E.g., to be a helpful assistant
- Solution: Fine-tune on a smaller, highly curated dataset after
 - These are examples you really do want the model to imitate
 - Pre-training has taught the model many things; fine-tuning tells it to only verbalize the parts that are desirable



Outline

- Why is fine-tuning not enough?
- Reinforcement Learning and Policy Gradient
- Using RL for LLM Post-Training
- Three common RL methods: GRPO, PPO, DPO*
 - (DPO is debatably not RL)

Limitations of Supervised fine-tuning

- Problem 1: Data scale
 - High-quality data is expensive to obtain, you don't have that much of it (relative to pretraining data)

Fine-Tuning Data Requirements

- Collect a dataset of (prompt, response) pairs
- Train model to maximize $P(\text{response} \mid \text{prompt})$
- Acquiring this data is not so easy!
 - Especially if you want your model to write long responses...

Problem

The battery charge in Mary's cordless vacuum cleaner lasts ten minutes. It takes her four minutes to vacuum each room in her house. Mary has three bedrooms, a kitchen, and a living room. How many times does Mary need to charge her vacuum cleaner to vacuum her whole house?

Solution

Mary has $3 + 1 + 1 = 5$ rooms in her house.

At 4 minutes a room, it will take her $4 * 5 = 20$ minutes to vacuum her whole house.

At 10 minutes a charge, she will need to charge her vacuum cleaner $20 / 10 = 2$ times to vacuum her whole house.

“Distant” Supervision

- For fine-tuning, we directly supervise the model on *exactly* what to output
 - But this takes work for us to write!
- What if we just *supervise the final answer*?
- Advantage: Now our dataset only needs to have (prompt, *final answer*) pairs = much easier to collect

Problem

The battery charge in Mary's cordless vacuum cleaner lasts ten minutes. It takes her four minutes to vacuum each room in her house. Mary has three bedrooms, a kitchen, and a living room. How many times does Mary need to charge her vacuum cleaner to vacuum her whole house?

Solution

Mary has 5 rooms in her house.
At 4 minutes a room, it takes 20 minutes to vacuum her whole house.
At 10 minutes, she will need to charge her vacuum cleaner $20 / 10 = 2$ times to vacuum her whole house.

Final Answer

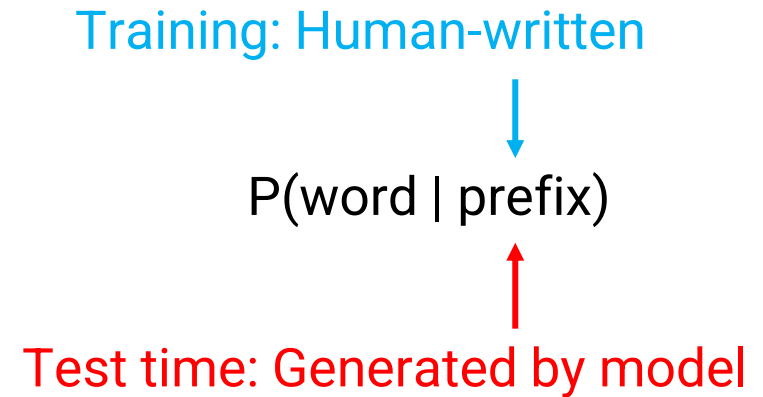
2

Limitations of Supervised fine-tuning

- Problem 1: Data scale
 - High-quality data is expensive to obtain, you don't have that much of it (relative to pretraining data)
- Problem 2: Exposure bias

Exposure bias

- Training time: Model learns to predict next word **given human-written prefix**
- Test time: Model must predict next word given **model-written prefix**
- **Exposure bias**: Model was never “exposed” to its own outputs during training, so it may not know what to do!



Limitations of Supervised fine-tuning

- Problem 1: Data scale
 - High-quality data is expensive to obtain, you don't have that much of it (relative to pretraining data)
- Problem 2: Exposure bias
- **Problem 3: Can promote guessing rather than factual responses**

Supervised Fine-tuning and Factuality

- Consider the following fine-tuning examples
 - Prompt: “When was the US Declaration of Independence signed?”
Answer: “July 4, 1776”
 - Model has probably seen this information many times during pre-training
 - So it has probably learned features that associate the Declaration of Independence with July 4
 - Prompt: “When was Robin Jia born?”
Answer: “[...]”
 - Model has probably (?) not seen this information during pre-training
 - Cannot have learned features associating me with my birthday
 - Supervised fine-tuning **encourages the model to just make something up!**

Limitations of Supervised fine-tuning

- Problem 1: **Data scale**
 - You simply don't have much data at your disposal
- Problem 2: **Exposure bias**
- Problem 3: **Can promote guessing rather than factual responses**
- Solution: **Fine-tune the language model with reinforcement learning!**
 - Encourages the **model's full outputs** to be **correct/factual**
 - Supervision signal is **computed based on rules or with another model** (can get "infinite" data now)

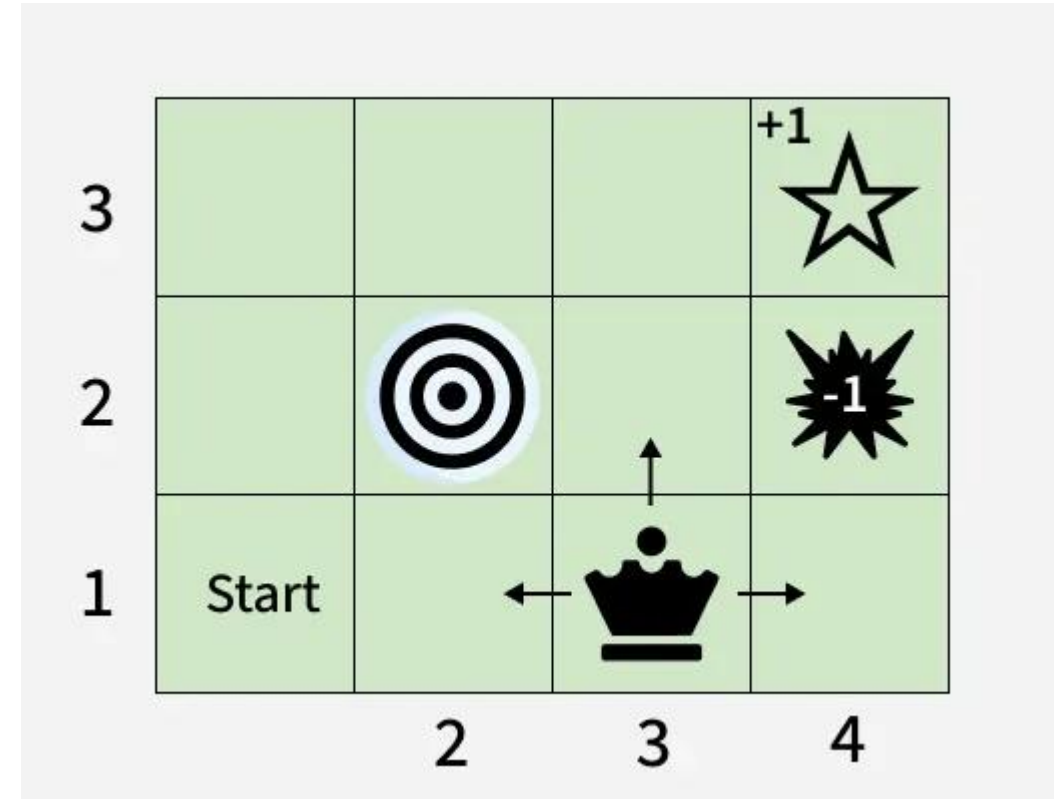
Outline

- Why is fine-tuning not enough?
- **Reinforcement Learning and Policy Gradient**
- Using RL for LLM Post-Training
- Three common RL methods: GRPO, PPO, DPO*
 - (DPO is debatably not RL)

Reinforcement Learning in a Nutshell

Typical RL problem

- Agent taking actions in an environment
- **State**: Current state of the world
- **Actions**: Things agent can do to change the world state
- **Rewards**: Received when some goal is achieved



Language Models & RL

Typical RL problem

- Agent taking actions in an environment
- **State:** Current state of the world
- **Actions:** Things agent can do to change the world state
- **Rewards:** Received when some goal is achieved

Language Modeling

- “Agent” is just the language model itself
- **State:** Input prompt + list of words generated so far
- **Actions:** Generate the next word in the response
- **Rewards:** Receive reward at the end if response was good
 - (Reward = 0 at middle timesteps)

Exposure bias and reinforcement learning

- In RL, supervised fine-tuning is called “**imitation learning**”
 - Known to be suboptimal due to exposure bias
 - You can try to mimic an expert player, but a worse player also needs to know how to recover from mistakes
- In real RL method, agent **takes its own actions**, ends up in some suboptimal state, and has to find the best path forward



Language Model as a Policy

- **Policy:** A (possibly stochastic) mapping from states to actions
 - For LM: The language model's $P(\text{next token} \mid \text{context})$ is a **policy** describing the likelihood of choosing any action (= token) in a given state (= context)
- What makes a policy good? **High expected reward**

$$\mathbb{E}_{y \sim p_{\theta}(\cdot | x)} [R(x, y)] = \sum_y p_{\theta}(y \mid x) \cdot R(x, y)$$

Sum over all
possible
responses y

Probability of y
under LM with
parameters θ ,
given prompt x

Reward for
outputting y
given prompt x

Policy Gradient

- How do we fine-tune θ to maximize expected reward? Gradient Ascent!
- Key question: How to compute gradient of the expected reward?

$$\nabla_{\theta} \sum_y p_{\theta}(y \mid x) \cdot R(x, y)$$

$$= \sum_y \nabla_{\theta} p_{\theta}(y \mid x) \cdot R(x, y)$$

Move gradient inside sum
Intractable sum over y , can't compute

$$= \sum_y p_{\theta}(y \mid x) \nabla_{\theta} \log p_{\theta}(y \mid x) \cdot R(x, y)$$

Small trick: $d(\log u)/dx = 1/u * du/dx$, so
 $du/dx = u * d(\log u)/dx$

$$= \mathbb{E}_{y \sim p_{\theta}(\cdot \mid x)} [\nabla_{\theta} \log p_{\theta}(y \mid x) \cdot R(x, y)]$$

Recognize as an expected value

Policy Gradient

- What's so great about $\mathbb{E}_{y \sim p_\theta(\cdot | x)} [\nabla_\theta \log p_\theta(y | x) \cdot R(x, y)]$?
- Since it's an expected value, we can approximate it by taking the average across some samples!
- Basic RL algorithm (“Policy Gradient” recipe):
 - Sample k responses $y_1, \dots, y_k$ from your current model (“rollouts”)
 - Compute rewards $R_1, \dots, R_k$ for them
 - Compute estimated gradient as $\frac{1}{k} \sum_{i=1}^k \nabla_\theta \log p_\theta(y_i | x) \cdot R_i$
 - Update θ in direction of the gradient
 - Repeat for new prompt x (or batch of prompts)

Outline

- Why is fine-tuning not enough?
- Reinforcement Learning and Policy Gradient
- **Using RL for LLM Post-Training**
- Three common RL methods: GRPO, PPO, DPO*
 - (DPO is debatably not RL)

RL with Verifiable Rewards (RLVR)

- Example: Learning to do math problems
- Here, we have a **verifiable reward**: Final answer correctness can be checked automatically
 - Reward = 1 if correct, 0 else
- RL training loop:
 - Get a batch of math problems
 - Do rollouts for each problem to get responses
 - Estimate expected reward gradient
 - Update parameters by taking a gradient step

Problem

The battery charge in Mary's cordless vacuum cleaner lasts ten minutes. It takes her four minutes to vacuum each room in her house. Mary has three bedrooms, a kitchen, and a living room. How many times does Mary need to charge her vacuum cleaner to vacuum her whole house?

Solution

Mary has 5 rooms in her house.
At 4 minutes a room, it takes 20 minutes to vacuum her whole house.
At 10 minutes, she will need to charge her vacuum cleaner $20 / 10 = 2$ times to vacuum her whole house.

Final Answer

2

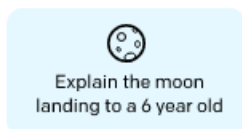
RL from Human Feedback (RLHF)

Step 1

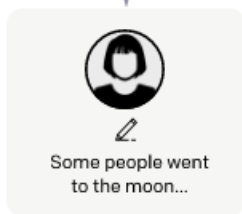
Collect demonstration data,
and train a supervised policy.

Supervised Fine-tuning

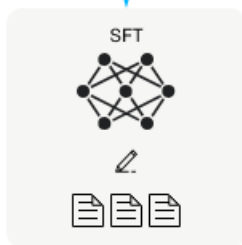
A prompt is
sampled from our
prompt dataset.



A labeler
demonstrates the
desired output
behavior.



This data is used
to fine-tune GPT-3
with supervised
learning.

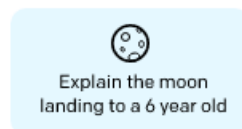


Step 2

Collect comparison data,
and train a reward model.

Learn to assign rewards

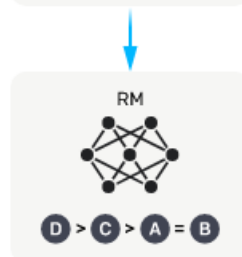
A prompt and
several model
outputs are
sampled.



A labeler ranks
the outputs from
best to worst.



This data is used
to train our
reward model.



Step 3

Optimize a policy against
the reward model using
reinforcement learning.

Do RL on learned rewards

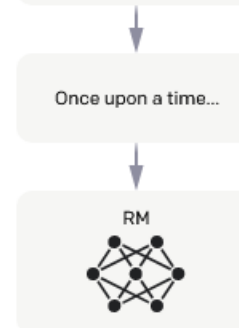
A new prompt
is sampled from
the dataset.



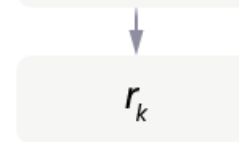
The policy
generates an output.



The reward model
calculates a
reward for
the output.



The reward is
used to update
the policy
using PPO.



- “RLHF” for short
- Trains language model with RL
- Rewards come from a model trained to predict human preferences

RLHF: The Data

Step 1

Collect demonstration data,
and train a supervised policy.

Supervised Fine-tuning

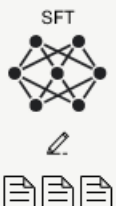
A prompt is
sampled from our
prompt dataset.

Explain the moon
landing to a 6 year old

A labeler
demonstrates the
desired output
behavior.

Some people went
to the moon...

This data is used
to fine-tune GPT-3
with supervised
learning.



Step 2

Collect comparison data,
and train a reward model.

Learn to assign rewards

A prompt and
several model
outputs are
sampled.

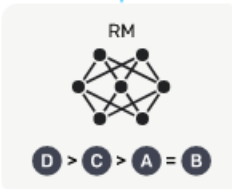
Explain the moon
landing to a 6 year old

Explain gravity... Explain war...
Moon is natural satellite of... People went to the moon...

A labeler ranks
the outputs from
best to worst.

D > C > A = B

This data is used
to train our
reward model.



Part 1: Prompts

- Some written by hired annotators
- Some from actual user queries to the OpenAI API

Part 2: Demonstrations

- Hired annotator writes a desired response to prompt

Part 3: Rankings

- Sample several model responses from model's probability distribution
- Hired annotator ranks them from best to worst

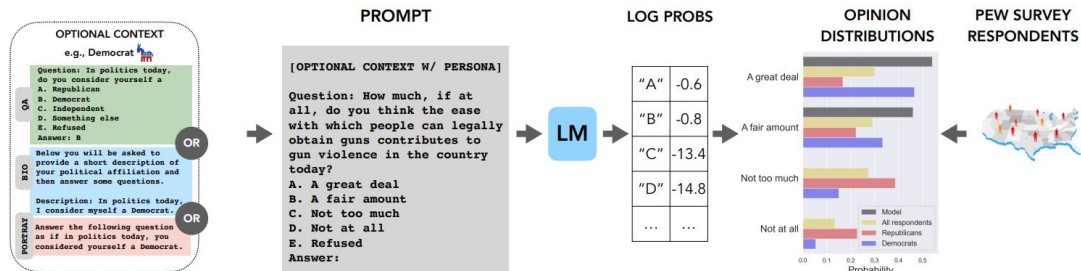
RLHF: Who's behind the data?

- InstructGPT paper (precursor to ChatGPT): “We hired a team of about 40 contractors on Upwork and through ScaleAI”
- Labelers had to pass various screening tests

What gender do you identify as?	
Male	50.0%
Female	44.4%
Nonbinary / other	5.6%
What ethnicities do you identify as?	
White / Caucasian	31.6%
Southeast Asian	52.6%
Indigenous / Native American / Alaskan Native	0.0%
East Asian	5.3%
Middle Eastern	0.0%
Latinx	15.8%
Black / of African descent	10.5%
What is your nationality?	
Filipino	22%
Bangladeshi	22%
American	17%
Albanian	5%
Brazilian	5%
Canadian	5%
Colombian	5%
Indian	5%
Uruguayan	5%
Zimbabwean	5%

What is your age?	
18-24	26.3%
25-34	47.4%
35-44	10.5%
45-54	10.5%
55-64	5.3%
65+	0%
What is your highest attained level of education?	
Less than high school degree	0%
High school degree	10.5%
Undergraduate degree	52.6%
Master's degree	36.8%
Doctorate degree	0%

RLHF: Whose opinions?



- Idea: Ask language models to answer pew research survey questions
- Models after RLHF agrees most with college-educated, Asian, 18-49 year olds
 - Matches InstructGPT demographics!
 - Non-RLHF models more similar to less educated respondents

	AI21 Labs			OpenAI					
Model	j1-grande	j1-jumbo	j1-grande-v2-beta	ada	davinci	text-ada-001	text-davinci-001	text-davinci-002	text-davinci-003
EDUCATION									
Less than high school	0.827	0.828	0.812	0.835	0.801	0.710	0.714	0.750	0.684
High school graduate	0.817	0.816	0.799	0.826	0.790	0.711	0.712	0.755	0.690
Some college, no degree	0.811	0.814	0.803	0.823	0.790	0.706	0.714	0.762	0.700
Associate's degree	0.809	0.811	0.800	0.821	0.789	0.703	0.712	0.761	0.699
College graduate/some postgrad	0.797	0.802	0.793	0.810	0.780	0.701	0.713	0.766	0.710
Postgraduate	0.788	0.794	0.789	0.800	0.775	0.695	0.712	0.766	0.716

Closest relative of ChatGPT

Highest similarity

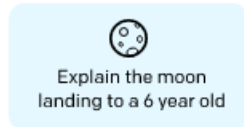
RLHF: Initial Supervised Fine-tuning

Step 1

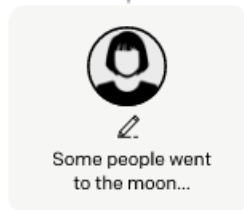
Collect demonstration data,
and train a supervised policy.

Supervised Fine-tuning

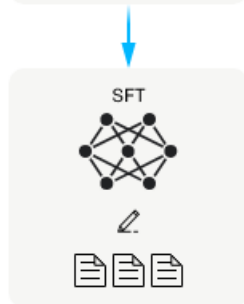
A prompt is
sampled from our
prompt dataset.



A labeler
demonstrates the
desired output
behavior.



This data is used
to fine-tune GPT-3
with supervised
learning.



- Run imitation learning on labeler-provided demonstrations, given prompt as prefix
- Very similar to pre-training, except:
 - Only compute loss on response tokens, not on prompt tokens
 - Much smaller dataset

Loss on this example =

$$\begin{aligned} & \log P(\text{"Some"} \mid \text{"Explain the moon landing to a 6 year old:"}) \\ & + \log P(\text{"people"} \mid \text{"Explain the moon landing to a 6 year old: Some"}) \\ & + \dots \end{aligned}$$

RLHF: The Reward Model

Step 2

Collect comparison data,
and train a reward model.

Learn to assign rewards

A prompt and
several model
outputs are
sampled.

🧠
Explain the moon
landing to a 6 year old

A Explain gravity...

B Explain war...

C Moon is natural
satellite of...

D People went to
the moon...

A labeler ranks
the outputs from
best to worst.

👤
D > C > A = B

This data is used
to train our
reward model.

RM
🧠
D > C > A = B

- Goal: Fine-tune LLM with RL
- Step 1: Show human annotators several sampled model outputs, ask them to rank
 - Provides some RL training data, but not a ton
- Step 2: Train a “**reward model**” to predict the human’s rankings
 - Now we can run as many RL training episodes as we want for free, using the reward model in place of the human annotators

RLHF: The Reward Model

Step 2

Collect comparison data,
and train a reward model.

Learn to assign rewards

A prompt and
several model
outputs are
sampled.

🧠
Explain the moon
landing to a 6 year old

A Explain gravity...

B Explain war...

C Moon is natural
satellite of...

D People went to
the moon...

A labeler ranks
the outputs from
best to worst.

👤
D > C > A = B

This data is used
to train our
reward model.

↓
RM
🧠
D > C > A = B

- Reward model
 - Start with a (separate) pretrained language model
 - Then fine-tune all parameters

RLHF: Training the Reward Model

Step 2

Collect comparison data,
and train a reward model.

Learn to assign rewards

A prompt and
several model
outputs are
sampled.

Explain the moon
landing to a 6 year old

A Explain gravity...
B Explain war...

C Moon is natural
satellite of...
D People went to
the moon...

A labeler ranks
the outputs from
best to worst.

D > C > A = B

This data is used
to train our
reward model.

RM
D > C > A = B

- Input: Prompt x , winning output y_w , losing output y_l
 - Wins/losses determined by labelers' rankings
- Reward model predicts scalar reward $r_\theta(x, y)$ given prompt x and model output y
- Objective on one example is to minimize:
$$-\log \sigma(r_\theta(x, y_w) - r_\theta(x, y_l))$$
 - Want reward on y_w to be higher than on y_l
 - Use the familiar logistic regression loss function!
 - Loss goes to 0 if argument is large
 - Loss goes to infinity if argument is small
 - Binary classification of which output is better

RLHF: Doing RL on the Language Model

Step 3

Optimize a policy against the reward model using reinforcement learning.

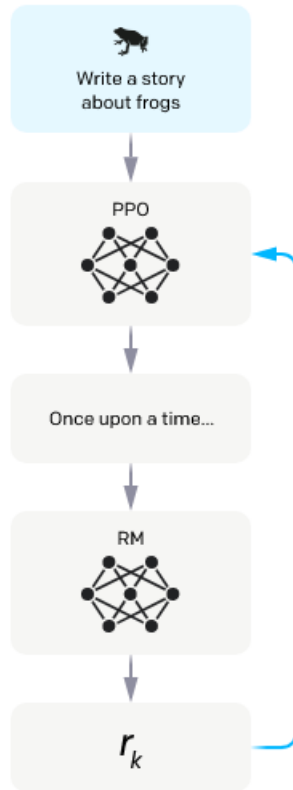
Do RL on learned rewards

A new prompt is sampled from the dataset.

The policy generates an output.

The reward model calculates a reward for the output.

The reward is used to update the policy using PPO.



- Use policy gradient method to **maximize expected reward** when **sampling** from LM's policy

RLHF and Factuality

- Let's revisit the earlier examples:
 - Prompt: "When was the US Declaration of Independence signed?"
Answer: "July 4, 1776"
 - Model outputs
 - "July 4, 1776" 
 - "January 1, 1950" 
 - "I don't know" 
- Top-ranked answer is the real answer

RLHF and Factuality

- Let's revisit the earlier examples:
 - Prompt: "When was Robin Jia born?"
Answer: "[...]"
 - Model outputs
 - "December 7, 1831"
 - "May 23, 1989"
 - "I don't know"
- Top-ranked answer is to say "I don't know"
- Overall effect: Model encouraged to say "I don't know" when its top candidate outputs are wrong



Outline

- Why is fine-tuning not enough?
- Reinforcement Learning and Policy Gradient
- Using RL for LLM Post-Training
- **Three common RL methods: GRPO, PPO, DPO***
 - (DPO is debatably not RL)

PPO & GRPO

- Proximal Policy Optimization (PPO): Came first, popular with RLHF
- Group Relative Policy Optimization (GRPO): Currently popular with RLVR
- They share three big ideas:
 - Optimize for “Advantage” (how much better is 1 response than average), rather than Reward
 - Consider divergence between “data collecting” and current policy
 - Prevent policy from drifting too much from original

Advantage in RL

- Original policy gradient estimate: $\frac{1}{k} \sum_{i=1}^k \nabla_{\theta} \log p_{\theta}(y_i | x) \cdot R_i$
- What if all R_i 's are very large?
 - We will get large push towards every y_i
 - y_i 's are sampled randomly, so there is huge variance from sampling!
- Idea: estimate advantage $A_i = \text{improvement of } y_i \text{ over the mean}$
 - If y_i is better than average, increase $p_{\theta}(y_i | x)$
 - If y_i is worse than average, decrease $p_{\theta}(y_i | x)$
- New formula: $\frac{1}{k} \sum_{i=1}^k \nabla_{\theta} \log p_{\theta}(y_i | x) \cdot A_i$

Advantage in GRPO

- Sample responses $y_1, \dots, y_k$
- Get rewards $R_1, \dots, R_k$
- Advantage for response i is simply:

$$A_i = \frac{R_i - \text{mean}(R_1, \dots, R_k)}{\text{stdev}(R_1, \dots, R_k)}$$

- English: Advantage is how many standard deviations the response is above/below the average response in our sample

Advantage in PPO

- GRPO: Advantage calculated as a sample mean over rollouts
 - Very simple
 - Requires many rollouts
- PPO:
 - Use a separate model to estimate the average reward
 - Part of a large family of methods called “actor-critic”
 - Actor = Policy (LLM)
 - Critic = Value estimator (this separate model)
 - Advantage: Don't need many rollouts per example anymore
 - Disadvantage: Now you have to train a second model in tandem

On-policy vs. Off-policy

- Recall our gradient expression: $\mathbb{E}_{y \sim p_{\theta}(\cdot | x)} [\nabla_{\theta} \log p_{\theta}(y | x) \cdot R(x, y)]$
- Our policy shows up twice here
 - #1: We sample responses y from our policy
 - #2: We compute gradient of our policy
- Ideally, these are always exactly the same policy (we sampled “**on-policy**”)
- But often times, the sampled responses come from an older policy (we sampled “**off-policy**”)
 - Sampling responses is expensive! We’d like to take multiple gradient steps per each sample
 - When training on large cluster, synchronization is a challenge

Correcting for being Off-Policy

- Luckily, there's a way to correct for being (slightly) off-policy
- Suppose we have samples from the model θ_{old}

$$\begin{aligned}\mathbb{E}_{y \sim p_{\theta}(\cdot | x)}[F(x, y)] &= \sum_y p_{\theta}(y | x) \cdot F(x, y) \\ \text{Goal: Compute expected value of any function under the current policy } \theta & \\ &= \sum_y p_{\theta_{old}}(y | x) \cdot \frac{p_{\theta}(y | x)}{p_{\theta_{old}}(y | x)} \cdot F(x, y) \\ &= \mathbb{E}_{y \sim p_{\theta_{old}}(\cdot | x)} \left[\frac{p_{\theta}(y | x)}{p_{\theta_{old}}(y | x)} \cdot F(x, y) \right] \\ &\quad \text{Expected value under old policy} \quad \text{Correction factor ("Importance weights")}\end{aligned}$$

Importance-weighted Advantage

- Putting these two ideas together, we want to maximize the **importance-weighted advantage**

Previous: Maximize Reward $\mathbb{E}_{y \sim p_{\theta}(\cdot|x)} [R(x, y)]$

New objective:

$$\mathbb{E}_{y \sim p_{\theta_{old}}(\cdot|x)} \left[\frac{p_{\theta}(y | x)}{p_{\theta_{old}}(y | x)} A(x, y) \right]$$

Clipping

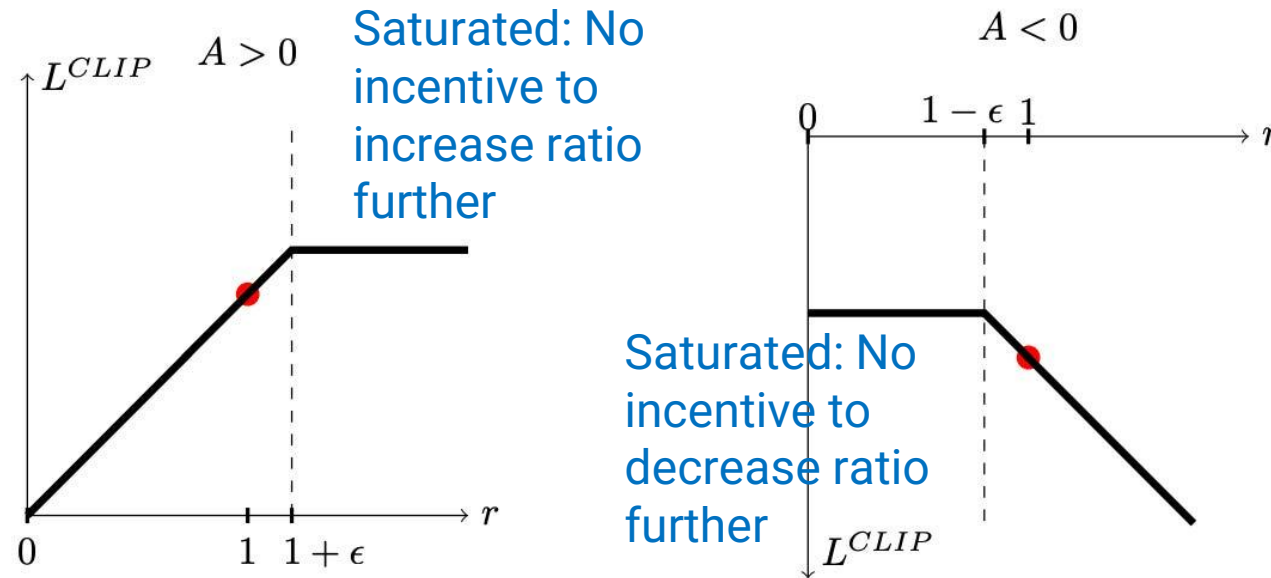
- Want to ensure our learned policy doesn't drift too far from the base policy
- Idea: We maximize a *clipped* objective:

$$\mathbb{E}_{y \sim p_{\theta_{old}}(\cdot|x)} \left[\min \left(\frac{p_{\theta}(y|x)}{p_{\theta_{old}}(y|x)} A(x, y), \text{clip} \left(\frac{p_{\theta}(y|x)}{p_{\theta_{old}}(y|x)}, 1 - \epsilon, 1 + \epsilon \right) A(x, y) \right) \right]$$

Looks familiar

If the probability ratio becomes too large/small, stop optimizing this

Clipping



- When $A > 0$: Only increase probability ratio up to $1 + \epsilon$
- When $A < 0$: Only decrease probability ratio down to $1 - \epsilon$

Summary: PPO & GRPO

- PPO & GRPO share three big ideas:
 - Optimize for “Advantage” (how much better is 1 response than average), rather than Reward
 - GRPO: Advantage estimated via sample mean/variance
 - PPO: Advantage estimated using a separate model
 - Use importance weights to adjust for divergence between “data collecting” and current policy
 - Prevent policy from drifting too much from original

Direct Preference Optimization (DPO)

- A completely different approach!
- Let's revisit RLHF
 - First we trained a reward model on some preference data
 - Then we trained our policy to optimize rewards given by the reward model
 - **Can we combine these into a single step???**

Step 2

Collect comparison data,
and train a reward model.

Learn to assign rewards

A prompt and
several model
outputs are
sampled.

Explain the moon
landing to a 6 year old

A B
Explain gravity... Explain war...
C D
Moon is natural satellite of... People went to the moon...

A labeler ranks
the outputs from
best to worst.

D > C > A = B

This data is used
to train our
reward model.

RM
D > C > A = B

Step 3

Optimize a policy against
the reward model using
reinforcement learning.

Do RL on learned rewards

A new prompt
is sampled from
the dataset.

Write a story
about frogs

The policy
generates
an output.

PPO

Once upon a time...

The reward model
calculates a
reward for
the output.

RM

The reward is
used to update
the policy
using PPO.

r_k

Property of the Optimal RL solution

- Under a particular RL setup:
 - We maximize rewards while also minimizing KL divergence between our current policy θ and old policy θ_{old} (regularization)
- Then the optimal policy θ satisfies:

$$R(x, y) = \beta \log \frac{p_{\theta}(y | x)}{p_{\theta_{old}}(y | x)} + \beta \log Z(x)$$

If reward of y is high,
optimal policy will put
proportionally higher
probability on it

- β is the hyperparameter controlling strength of KL regularization
- $Z(x)$ is some normalization constant

Deriving the DPO Loss

- Reward Model: For winning response y_w & losing response y_l , we wanted to minimize the loss

$$-\log \sigma(r_\theta(x, y_w) - r_\theta(x, y_l))$$

- Let's plug in our expression for the reward!

$$R(x, y) = \beta \log \frac{p_\theta(y|x)}{p_{\theta_{old}}(y|x)} + \beta \log Z(x)$$

- Result: Full DPO loss:

$$-\log \sigma \left(\beta \log \frac{p_\theta(y_w | x)}{p_{\theta_{old}}(y_w | x)} - \beta \log \frac{p_\theta(y_l | x)}{p_{\theta_{old}}(y_l | x)} \right)$$

- We train our policy parameters directly on the human preference data!

DPO vs. “Real” RL methods

- PPO and GRPO are real RL methods
 - The policy actually “takes actions” in the environment (does rollouts)
- DPO uses some RL theory to derive a principled learning objective, but is not a (typical) RL method itself
 - Vanilla DPO does not involve any rollouts—very off-policy
 - Can do sequential DPO: Run DPO once, generate responses, get those human-labeled, Run DPO again, ...

Conclusion

- Post-training helps us get the desired behavior out of the pre-trained model
- Reinforcement learning complements supervised fine-tuning